

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 20
Graphs (I)

Data Structures and Algorithms

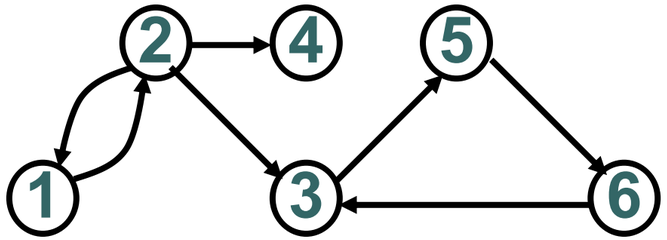
Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Graphs: Definitions

- A graph *G* consists of a set *V*, whose members are called the vertices of *G*, together with a set *E* of pairs of distinct vertices from *V* .
- The pairs in *E* are called the edges of *G*.
- If *e* = (*v*, *w*) is an edge with vertices *v* and *w*, then *v* and *w* are said to lie on *e*, and *e* is said to be incident with (邻接) *v* and *w*.
- If the pairs are unordered, *G* is called an undirected graph.
- If the pairs are ordered, G is called a directed graph. The term directed graph is often shortened to digraph, and the unqualified term *graph* usually means *undirected graph*.
- Two vertices in an undirected graph are called adjacent (相邻的) if there is an edge from the first to the second.

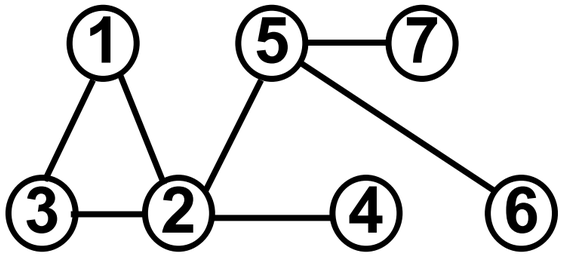
Examples of Graphs



G_1

$V(G_1)=\{1,2,3,4,5,6\}$

$E(G_1)=\{<1,2>, <2,1>, <2,3>, <2,4>, <3,5>, <5,6>, <6,3>\}$



G_2

$V(G_2)=\{1,2,3,4,5,6,7\}$

$E(G_1)=\{(1,2), (1,3), (2,3), (2,4),(2,5), (5,6), (5,7)\}$

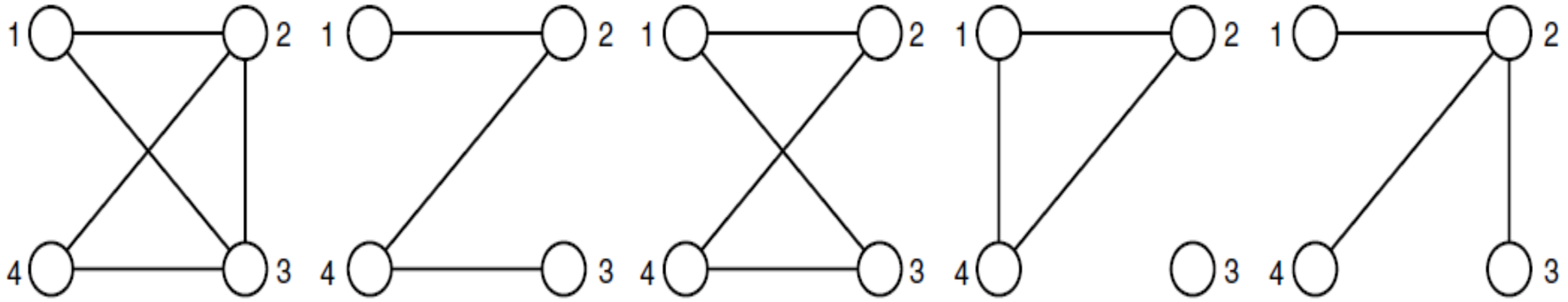
Graphs: Definitions

- A path is a sequence of distinct vertices, each adjacent to the next.
- A cycle is a path containing at least three vertices such that the last vertex on the path is adjacent to the first.
- A graph is called connected (连通的) if there is a path from any vertex to any other vertex.
- A free tree is defined as a connected undirected graph with no cycles.
- In a directed graph a path or a cycle means always moving in the direction indicated by the arrows. Such a path (cycle) is called a directed path (cycle).

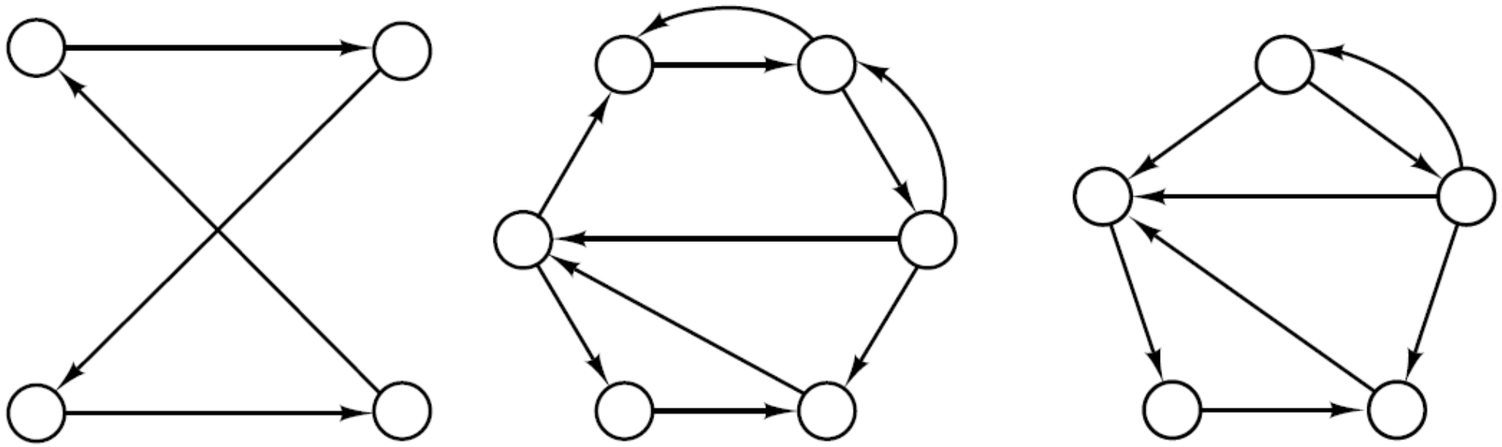
Graphs: Definitions

- A directed graph is called strongly connected (强连通) if there is a directed path from any vertex to any other vertex. If we suppress the direction of the edges and the resulting undirected graph is connected, we call the directed graph weakly connected (弱连通).
- The valence (or degree) of a vertex is the number of edges on which it lies, hence also the number of vertices adjacent to it.

Examples of Graphs



Connected (a) | Path (b) | Cycle (c) | Disconnected (d) | Tree (e)



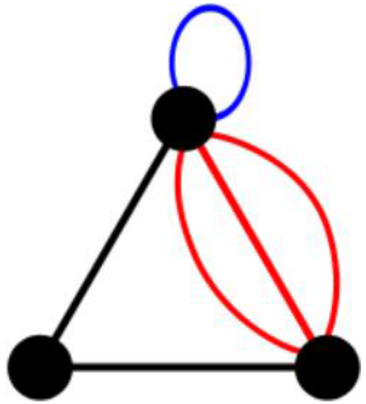
Directed cycle (a) | Strongly connected (b) | Weakly connected (c)

Graphs: Definitions

- A complete graph is a simple undirected graph in which every pair of distinct vertices is connected by a unique edge. Number of edges: n(n-1)/2.
- A complete digraph is a directed graph in which every pair of distinct vertices is connected by a pair of unique edges (one in each direction). Number of edges: n(n-1).
- A dense graph (稠密图) is a graph in which the number of edges is close to the maximal number of edges.
- A sparse graph (稀疏图) is a graph with only a few edges.
- Weighted graph (带权图) and Unweighted graph (无权图): In weighted graphs, each edge (or vertex) of G is assigned a numerical value, called weight or cost.

Graphs: Definitions

- An edge (x, y) is a *multi-edge* if it occurs more than once in the graph.
- Any graph which avoids these structures is called simple graph (简单图).



- An acyclic graph does not contain any cycles. Trees are connected acyclic undirected graphs.
- Directed acyclic graphs (有向无环图) are called DAGs.

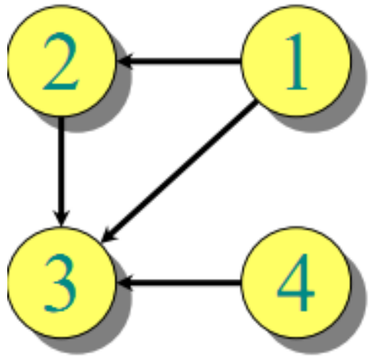
Graph Representations

- There are two main data structures used to represent graphs.
  - Adjacency Matrix (邻接矩阵)
  - Adjacency Lists (邻接表)
  - *Chained Forward Start* (链式前向星)*
- We assume the graph $G = (V, E)$ contains $n = |V|$ vertices and $m = |E|$ edges.
- Assume that the graph is a simple graph. (No multi-edges or self-loops)

Adjacency Matrix Representation

- The adjacency matrix of a graph $G = (V, E)$, where $V = \{1,2,\ldots,n\}$, is the matrix $A[1\ldots n, 1\ldots n]$ given by

$$A[i,j] = \begin{cases} 1 & \text{if } (i,j) \in E, \\ 0 & \text{if } (i,j) \notin E. \end{cases}$$



A	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	0	0	1	0

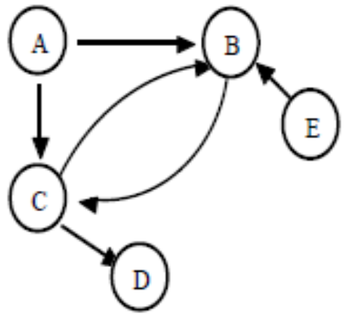
$\Theta(V^2)$ storage
$\Rightarrow$ ***dense***
representation.

- The matrix is symmetric (对称的) for undirected graphs
- degree(i) = number of 1s in the i-th row

Adjacency Matrix Representation

```
int n;              // num of vertex
int m;              // num of edges
int u, v;           // vertex of edge (u,v)
cin >> n >> m;
int g[n+1][n+1];   //store the graph
memset(g, 0, sizeof(g));
for (int j = 1; j <= m; j++)  {
    cin >> u >> v;
    g[u][v] = 1;
    if (directed==false) g[v][u] = 1;
}
```

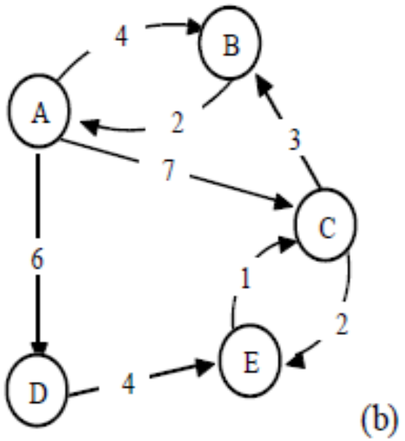
- Adjacency matrix representation for weighted graphs



(a)

	A	B	C	D	E
A	-1	1	1	-1	-1
B	-1	-1	1	-1	-1
C	-1	1	-1	1	-1
D	-1	-1	-1	-1	-1
E	-1	1	-1	-1	-1

-1 → 0



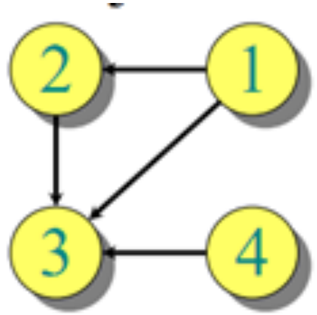
(b)

	A	B	C	D	E
A	-1	4	7	6	-1
B	2	-1	-1	-1	-1
C	-1	3	-1	-1	2
D	-1	-1	-1	-1	4
E	-1	-1	1	-1	-1

-1 → +∞

Adjacency List Representation

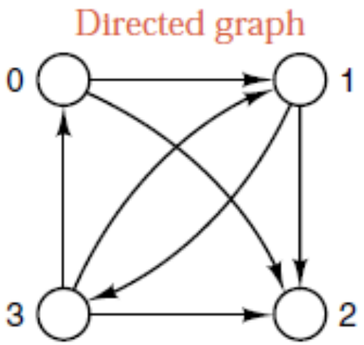
- An adjacency list of a vertex $v \in V$ is the list *Adj[v]* of vertices adjacent to *v*.



$Adj[1] = \{2, 3\}$
$Adj[2] = \{3\}$
$Adj[3] = \{\}$
$Adj[4] = \{3\}$

- For undirected graphs, |*Adj[v]*| = *degree(v)*
- For digraphs, |*Adj[v]*| = *out-degree(v)*

Adjacency List Representation



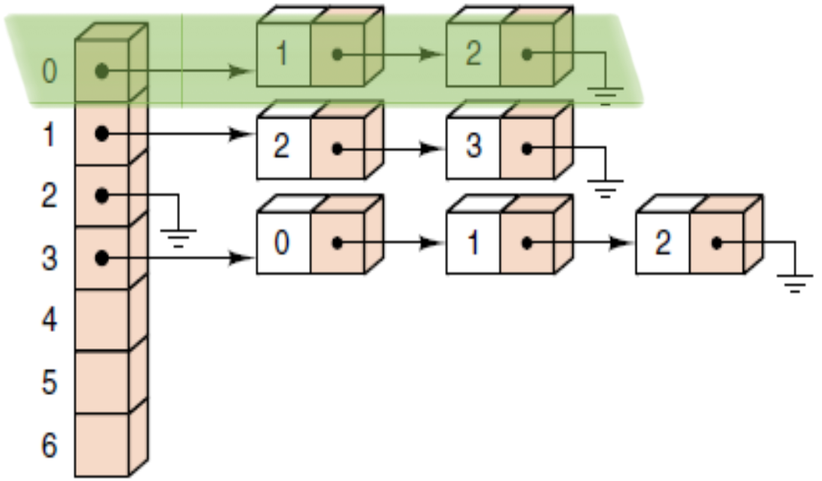
count = 4

vertex	adjacency list
0	1 2 – – – – –
1	2 3 – – – – –
2	– – – – – – –
3	0 1 2 – – – –
4	– – – – – – –
5	– – – – – – –
6	– – – – – – –

(b) Contiguous lists

count = 4

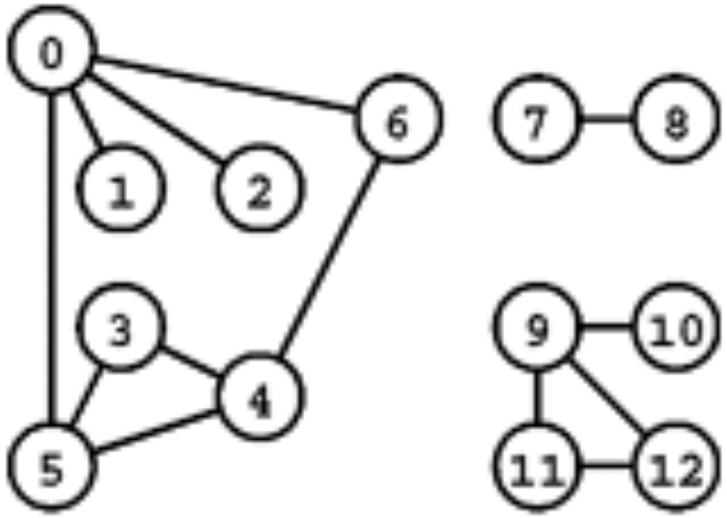
first_edge

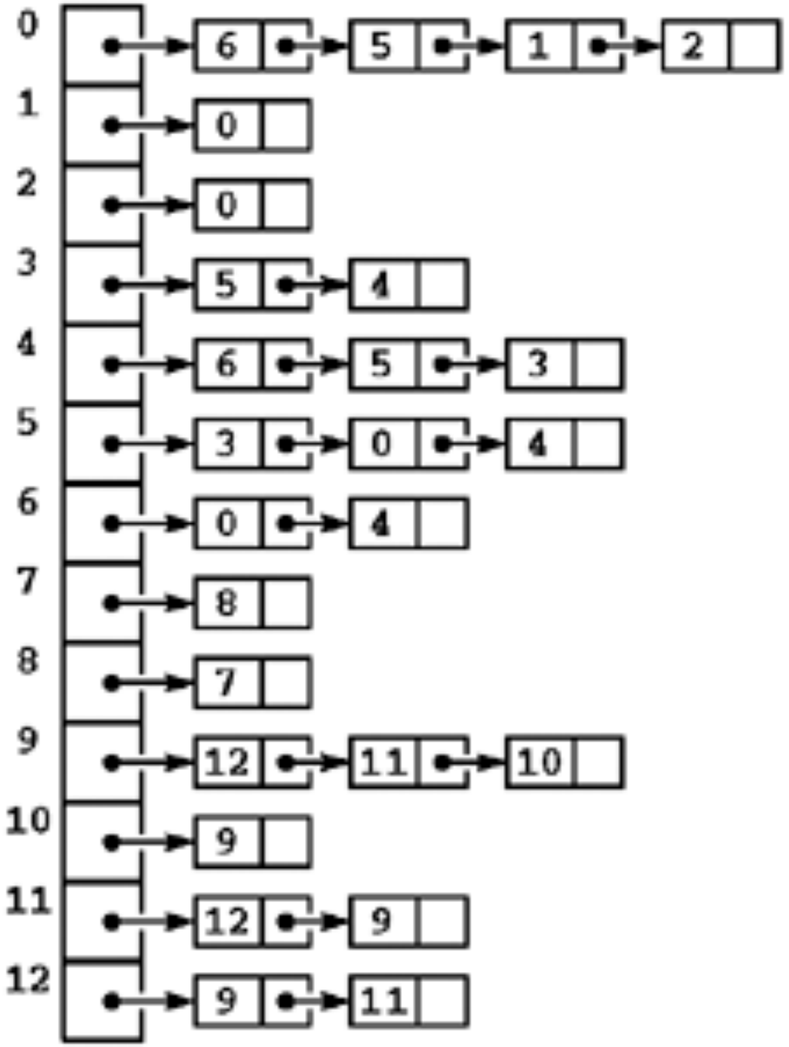


(c) Mixed

Adjacency List Representation

- Use Linked list





Adjacency List Representation

- Use STL list

```
int n; //num of vertex
int m; //num of edges
int u, v; // vertex of edge (u,v)
cin >> n >> m;
list<int> g[n+1]; //store the graph
for(int j=1; j<=m; j++)
{
      cin >> u >> v;
      g[u].push_back(v);
      if (directed==false) g[v].push_back(u);
}
```

What if use vector?

Analysis of Adjacency List Representation

- Handshaking Lemma: $\sum_{v \in V} d(v) = 2|E|$
- 对于n个顶点e条边的无向图，邻接表中除了n个头结点外，只有2e个表结点,空间效率为O(n+2e)。若是稀疏图($e<<n^2$)，则比邻接矩阵表示法$O(n^2)$省空间。
- 在有向图中，邻接表中除了n个头结点外，只有e个表结点,空间效率为O(n+e)。若是稀疏图，则比邻接矩阵表示法合适。
- **邻接表的优点**：空间效率高；容易寻找顶点的邻接点；
- **邻接表的缺点**：判断两顶点间是否有边或弧，需搜索两结点对应的单链表，没有邻接矩阵方便。

Comparisons between Adjacency Lists and Matrices

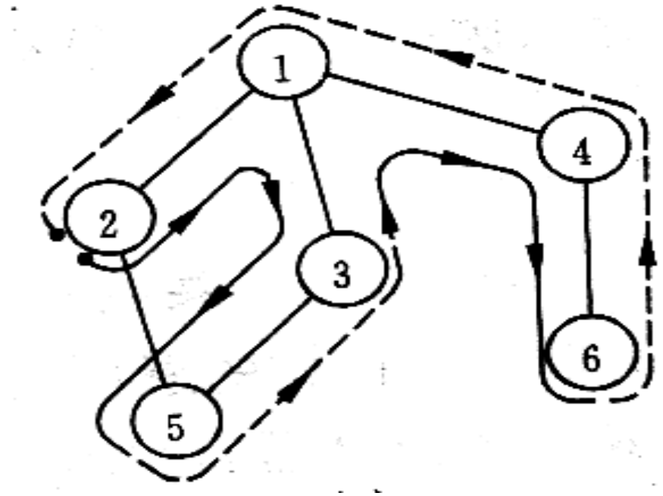
Comparison	Winner
Faster for testing if (x,y) exists?	matrices
Faster for finding vertex degree?	lists
Less memory on sparse graphs?	lists O(m+n) vs. $O(n^2)$
Less memory on dense graphs?	matrices
Easier for edge insertion/deletion?	matrices O(1) vs O(d)
Faster for traversing the graph?	lists O(m+n) vs. $O(n^2)$
Better for most problems?	lists

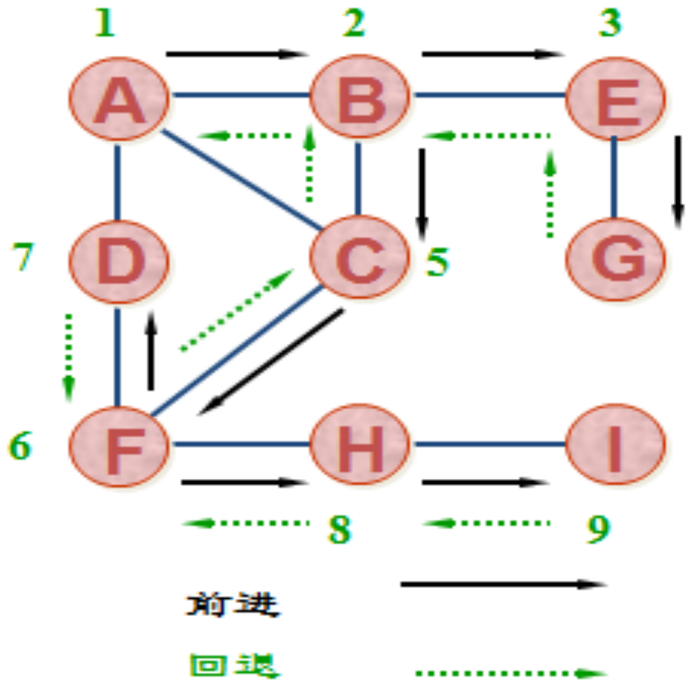
Graph Traversal

- One of the most fundamental graph problems is to traverse every edge and vertex in a graph.
  - Efficiency: visit each edge at most twice.
  - Correctness: do the traversal in a systematic way so that we don't miss anything.
- The key idea is that we must mark each vertex when we first visit it, and keep track of what have not yet completely explored.
- There are two traversal strategies
  - Depth-first
  - Breadth-first

Depth-first Traversal

- It is roughly analogous to preorder traversal of an ordered tree. Suppose that the traversal has just visited a vertex *v*, and let $w_1, w_2, \ldots, w_k$ be the vertices adjacent to *v*. Then we shall next visit w_1 and keep $w_2, \ldots, w_k$ waiting. After visiting w_1, we traverse all the vertices to which it is adjacent before returning to traverse $w_2, \ldots, w_k$.





Depth-first Traversal

- In case there are vertices which have not been visited, then chose one vertex and start depth first traversal until all vertices are visited.

```
template <int max_size>
void Digraph<max_size> :: depth_first(void (*visit)(Vertex &)) const
/* Post: The function *visit has been performed at each vertex of the Digraph in
         depth-first order.
   Uses: Method traverse to produce the recursive depth-first order. */
{
   bool visited[max_size];
   Vertex v;
   for (all v in G) visited[v] = false;
   for (all v in G) if (!visited[v])
      traverse(v, visited, visit);
}
```

Select the non-visited node

Depth-first Traversal

- The recursion is performed in an auxiliary function traverse.

```
template <int max_size>
void Digraph<max_size> :: traverse(Vertex &v, bool visited[ ],
                                   void (*visit)(Vertex &)) const
/* Pre:   v is a vertex of the Digraph.
   Post:  The depth-first traversal, using function *visit, has been completed for v and
          for all vertices that can be reached from v.
   Uses: traverse recursively.  */
{
   Vertex w;
   visited[v] = true;
   (*visit)(v);
   for (all w adjacent to v)
     if (!visited[w])
        traverse(w, visited, visit);
}
```

The vertex is marked as visited

Marking Vertices

- 任意结点总处于以下状态中的一种:
  - undiscovered – 所有结点的初始状态.
  - discovered – 结点第一次被访问，但它的出边未处理完毕。
  - processed – 结点的所有出边都已经处理完毕。

```
template <int max_size>
void Digraph<max_size> :: traverse(Vertex &v, bool visited[ ],
                                   void (*visit)(Vertex &)) const
/* Pre:  v is a vertex of the Digraph.
   Post: The depth-first traversal, using function *visit, has been completed for v and
         for all vertices that can be reached from v.
   Uses: traverse recursively.  */
{
   Vertex w;
   visited[v] = true;
   (*visit)(v);
   for (all w adjacent to v)
     if (!visited[w])
        traverse(w, visited, visit);
}
```

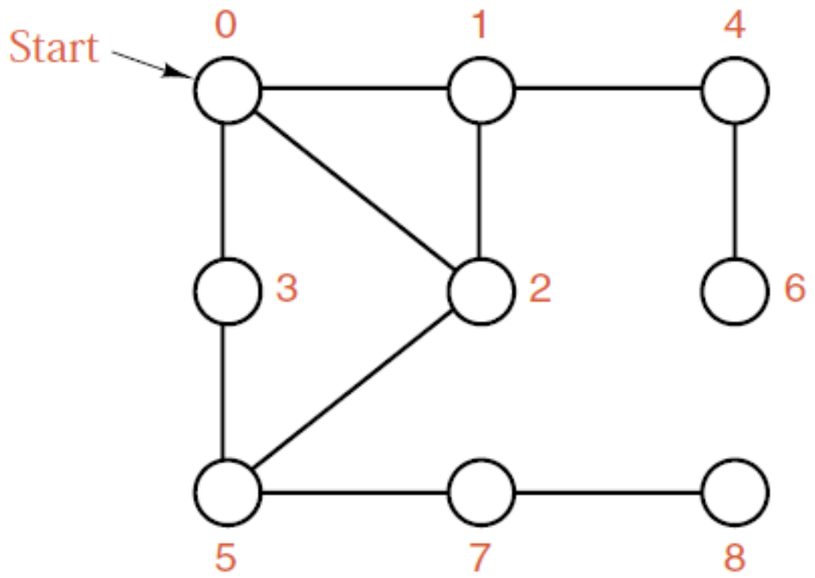
The vertex v is undiscovered before

The vertex v is discovering

The vertex v is processed

Breadth-first Traversal

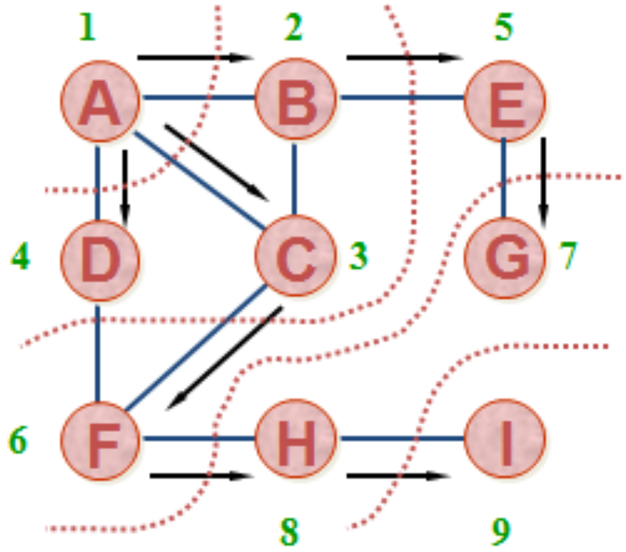
- It is roughly analogous to level-by-level traversal of an ordered tree. If the traversal has just visited a vertex *v,* then it next visits all the vertices adjacent to *v*, putting the vertices adjacent to these in a waiting list to be traversed after all vertices adjacent to *v* have been visited.



Breadth-first traversal

Breadth-first Traversal

- Breadth first traversal can be done by using a queue. The head is the vertex to be visited. When it is visited, all its adjacent vertices are put in the queue.
- Let v be the first vertex to be visited, and w_1, …, w_k be the adjacent vertices of v. We will first put v in the queue. Visit v and put w_1, …, w_k in the queue. Then visit w_1 and put the adjacent vertices of w_1 into the queue, …, until all the vertices have been visited.



Breadth-first Traversal

```
template <int max_size>
void Digraph<max_size> :: breadth_first(void (*visit)(Vertex &)) const
/* Post: The function *visit has been performed at each vertex of the Digraph in
         breadth-first order.
   Uses: Methods of class Queue.  */
{
   Queue q;
   bool visited[max_size];
   Vertex v, w, x;
   for (all v in G) visited[v] = false;
   for (all v in G)
     if (!visited[v]) {
        q.append(v);
        while (!q.empty()){
           q.retrieve(w);
           if (!visited[w]) {
              visited[w] = true;
              (*visit)(w);
              for (all x adjacent to w)
                 q.append(x);
           }
           q.serve();
        }
     }
}
```

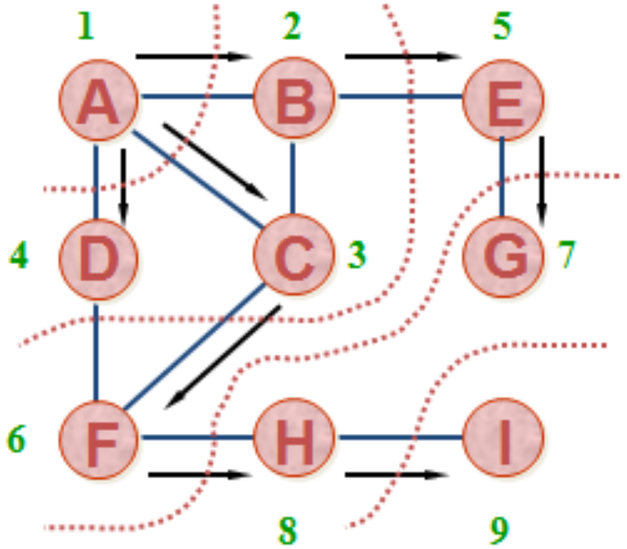
The vertex w is undiscovered before

The vertex w is discovering

The vertex w is processed

Finding Paths

- The parent array set within BFS() is very useful for finding interesting paths through a graph.
- The vertex which discovered vertex *i* is defined as parent[*i*].
- The parent relation defines a tree of discovery with the initial search node as the root of the tree.



vertex	A	B	C	D	E	F	G	H	I
Index	1	2	3	4	5	6	7	8	9
parent	-1	1	1	1	2	3	5	6	8

BFS Application

- Shortest Paths in Unweighted Graph
- In BFS, vertices are discovered in the order of increasing distance from the root, so this tree has a very important property.
- The unique tree path from the root to any node x∈ V uses the smallest number of edges (or equivalently, intermediate nodes) possible on any root-to-x path in the graph.

```
q.append(v);
visited[v] = true;
d[v] = 0;
parent[v] = -1;
while (!q.empty()) {
    q.retrieve(w);
    visited[w] = true;
    (*visit)(w);
    for (all x adjacent to w)
    if (!visited[x]) {
        q.append(x);
        d[x] = d[w] + 1;
        parent[x] = w;
    }
    q.serve();
}
```

d[v]: level of v

trace parent[] to discover a path from the root to a node

BFS Application

- Connected Components
- The *connected components* of an undirected graph are the separate “pieces” of the graph such that there is no connection between the pieces.
- Anything we discover during a BFS must be part of the same connected component. We then repeat the search from any undiscovered vertex (if one exists) to define the next component, until all vertices have been found.

Thank you!

